

PROYECTO ACADÉMICO — PROCESAMIENTO DE IMÁGENES

Resumen General, Justificación de Librerías y Análisis Línea por Línea (Ejercicios 4 al 7)

1. Resumen General de Cada Ejercicio

Ejercicio 4: Composición con Plantillas Geométricas

Objetivo: Superponer la imagen de Lena (*fig_00.jpg*) sobre 4 imágenes de fondo distintas (*fig_01.jpg* a *fig_04.jpg*) utilizando plantillas geométricas (círculo, rectángulo, pentágono y corazón; *pla_01.jpg* a *pla_04.jpg*) como máscaras de composición.

Mecanismo: Se redimensiona la imagen a superponer y la máscara al tamaño exacto de cada fondo. Mediante `Image.composite(sup, fondo, mascara)`, los píxeles blancos (255) de la plantilla dejan ver a Lena, los negros (0) dejan ver el fondo, y los bordes grises se difuminan suavemente sin distorsión.

Ejercicio 5: Separación de Planos R, G, B y Cálculo de Área

Objetivo: Descomponer la imagen de fluorescencia de neuronas (*fig_05.jpg*) en sus tres canales fundamentales Rojo, Verde y Azul, calculando el área física ocupada por cada componente de color.

Mecanismo: Con `img.split()` se extraen las bandas individuales. Se transforman en matrices NumPy y se aplica un umbral de corte (intensidad > 30) para aislar la emisión biológica del ruido de fondo. Se cuantifican los píxeles activos y su porcentaje respecto al área total, identificando el canal cromático predominante.

Ejercicio 6: Histograma RGB y Escala de Grises

Objetivo: Analizar la distribución estadística de intensidades lumínicas y cromáticas en la imagen de Lena (*fig_00.jpg*), extrayendo la tonalidad modal (más repetida) en cada canal y comparando el histograma en escala de grises.

Mecanismo: Se extrae el histograma de 768 posiciones con `img.histogram()`, dividiéndolo en tramos de 256 niveles para R, G y B. Con `np.argmax()` se localiza la moda exacta de cada canal. La imagen se convierte a escala de grises ('L') mediante la fórmula estándar de luminancia ITU-R 601-2, visualizando subplots con curvas individuales y acumulativas.

Ejercicio 7: Colorización de Imagen en Escala de Grises

Objetivo: Transformar una imagen monocromática de olas del mar (*sea.jpg*) en una imagen cromática vívida utilizando cuatro técnicas de colorización avanzadas.

Mecanismo: Se implementan cuatro algoritmos: 1) *Mapeo oceánico no lineal* en NumPy combinando funciones polinómicas y cúbicas; 2) *Tabla de búsqueda (LUT)* para efecto atardecer cálido con `Image.point()`; 3) *Paleta predefinida* mediante mapas de color `colormap` de Matplotlib ('ocean'); y 4) *Colorización tricromática* con `ImageOps.colorize()` complementada con realce de saturación y filtro de suavizado.

2. Librerías Utilizadas y Justificación Técnica

Librería / Módulo	Funciones Principales Usadas	Justificación de Uso en el Proyecto
PIL / Pillow (Image)	open(), resize(), convert(), split(), merge(), composite(), point(), save()	Librería estándar para I/O y procesamiento de imágenes en Python. Permite manipular formatos JPG/PNG, transformar espacios de color (RGB, L), recomponer imágenes con máscaras alfa y redimensionar con filtros de alta calidad como LANCZOS.
PIL / Pillow (ImageOps, ImageFilter, ImageEnhance)	ImageOps.colorize(), ImageFilter.SMOOTH, ImageEnhance.Color()	Módulos avanzados de postprocesamiento en el Ejercicio 7. colorize asigna gradientes tricromáticos (sombras/medios tonos/luces), SMOOTH elimina el banding/ruido del mar y Color amplifica la saturación un 30%.
NumPy (numpy)	array(), sum(), mean(), std(), argmax(), clip(), power(), stack()	Motor de cálculo matricial ultrarrápido en C. Permite tratar imágenes como tensores numéricos, vectorizar cálculos sobre millones de píxeles, segmentar áreas por umbralización booleana y diseñar curvas de transferencia de color matemáticas.
Matplotlib (pyplot)	subplots(), imshow(), bar(), plot(), axvline(), savefig()	Generación de visualizaciones comparativas, subplots estructurados, histogramas de frecuencia, gráficos de barras de áreas y mapas de falso color (colormaps científicos). Permite exportar resultados en alta resolución (DPI=150).
os	path.dirname(), path.abspath(), path.join()	Garantiza portabilidad multiplataforma (Windows/Linux/macOS) resolviendo rutas relativas al script en ejecución sin depender del directorio de trabajo actual.

3.1 Ejercicio 4 — Composición con Plantillas Geométricas

Archivo fuente: `ejercicio_4.py` (Código limpio sin comentarios)

Bloque 1: Importación de Librerías

```
1 | from PIL import Image
2 | import matplotlib.pyplot as plt
3 | import os
```

Explicación línea por línea:

- **Línea 1:** Importa el módulo central Image de Pillow para manipular, convertir y componer imágenes.
- **Línea 2:** Importa la interfaz pyplot de Matplotlib para la visualización tabular de resultados.
- **Línea 3:** Importa el módulo os para la gestión dinámica y segura de rutas de archivos en el sistema operativo.

Bloque 2: Definición y Configuración de Rutas

```
5 | directorio = os.path.dirname(os.path.abspath(__file__))
7 | ruta_superponer = os.path.join(directorio, "fig_00.jpg")
9 | rutas_fondo = [
10 |     os.path.join(directorio, "fig_01.jpg"),
11 |     os.path.join(directorio, "fig_02.jpg"),
12 |     os.path.join(directorio, "fig_03.jpg"),
13 |     os.path.join(directorio, "fig_04.jpg"),
14 | ]
16 | rutas_plantillas = [
17 |     os.path.join(directorio, "pla_01.jpg"),
18 |     os.path.join(directorio, "pla_02.jpg"),
19 |     os.path.join(directorio, "pla_03.jpg"),
20 |     os.path.join(directorio, "pla_04.jpg"),
21 | ]
23 | nombres_plantillas = ["Círculo", "Rectángulo", "Pentágono", "Corazón"]
25 | nombres_fondo = ["Libros con planta", "Universidad", "Libro en pasto", "Libro fantástico"]
```

Explicación línea por línea:

- **Línea 5:** Obtiene la ruta absoluta del directorio que contiene el script para garantizar portabilidad.
- **Línea 7:** Define la ruta de la imagen que será superpuesta (Lena: *fig_00.jpg*).
- **Línea 9-14:** Lista con las 4 rutas absolutas de los fondos sobre los cuales se proyectará Lena.
- **Línea 16-21:** Lista con las 4 rutas de las plantillas en blanco y negro (máscaras de recorte).
- **Línea 23, 25:** Listas de cadenas de texto descriptivas para las figuras geométricas y fondos, empleadas en títulos y nombres de archivo.

Bloque 3: Carga de Imagen a Superponer y Preparación de la Figura

```

27 | print("=" * 60)
28 | print(" EJERCICIO 4: Composición con Plantillas Geométricas")
29 | print("=" * 60)
30 |
31 | img_superponer = Image.open(ruta_superponer).convert("RGB")
32 | print(f"\nImagen a superponer: fig_00.jpg")
33 | print(f"  Tamaño: {img_superponer.size}")
34 | print(f"  Modo: {img_superponer.mode}")
35 |
36 | fig, axes = plt.subplots(4, 4, figsize=(18, 18))
37 | fig.suptitle("Ejercicio 4: Composición de Imágenes con Plantillas Geométricas",
38 |             fontsize=16, fontweight='bold', y=0.98)

```

Explicación línea por línea:

- **Línea 27-29:** Imprime en consola un encabezado formateado delimitado por líneas de 60 caracteres.
- **Línea 31:** Abre la imagen *fig_00.jpg* y asegura su representación en espacio de color de 3 canales RGB.
- **Línea 32-34:** Imprime información de control técnico: nombre, dimensiones en píxeles (ancho x alto) y modo de color.
- **Línea 36-38:** Inicializa una cuadrícula de 4x4 subplots (16 gráficos) en Matplotlib con tamaño 18x18 pulgadas y título principal negrita.

Bloque 4: Bucle Principal de Composición y Redimensionamiento

```

40 | for i in range(4):
41 |     img_fondo = Image.open(rutas_fondo[i]).convert("RGB")
42 |     img_plantilla = Image.open(rutas_plantillas[i]).convert("L")
43 |     tamaño = img_fondo.size
44 |     img_sup_resized = img_superponer.resize(tamaño, Image.LANCZOS)
45 |     mascara_resized = img_plantilla.resize(tamaño, Image.LANCZOS)
46 |     resultado = Image.composite(img_sup_resized, img_fondo, mascara_resized)

```

Explicación línea por línea:

- **Línea 40:** Itera sobre los 4 casos de composición (índices 0 a 3).
- **Línea 41:** Carga el fondo correspondiente y lo convierte a modo RGB.
- **Línea 42:** Carga la plantilla geométrica y la convierte a escala de grises ('L'), modo requerido para operar como máscara alfa.
- **Línea 43:** Extrae la tupla (ancho, alto) del fondo como resolución maestra de destino.
- **Línea 44-45:** Redimensiona la imagen de Lena y la plantilla geométrica al tamaño exacto del fondo usando el filtro *Image.LANCZOS* para máxima nitidez sin pixelado.
- **Línea 46:** Aplica el algoritmo *Image.composite*: los píxeles donde la máscara es 255 (blanco) provienen de Lena; donde es 0 (negro), provienen del fondo.

Bloque 5: Guardado en Disco y Presentación Gráfica

```

48 |     print(f"\n--- Composición {i+1}: Plantilla {nombres_plantillas[i]} ---")
53 |     nombre_salida = f"resultado_ej4_{nombres_plantillas[i].lower()}.jpg"
54 |     ruta_salida = os.path.join(directorio, nombre_salida)
55 |     resultado.save(ruta_salida, quality=95)
58 |
    axes[i, 0].imshow(img_fondo); axes[i, 0].set_title(f"Fondo: {nombres_fondo[i]}"); axes[i, 0].axis('off')
61 |     axes[i, 1].imshow(img_plantilla, cmap='gray'); axes[i, 1].set_title(f"Plantilla: {nombres_plantillas[i]}"); axes[i, 1].axis('off')
64 |
    axes[i, 2].imshow(img_sup_resized); axes[i, 2].set_title("Superponer: Lena"); axes[i, 2].axis('off')
67 |     axes[i, 3].imshow(resultado); axes[i, 3].set_title(f"Resultado: {nombres_plantillas[i]}"); axes[i, 3].axis('off')
72 | plt.tight_layout(rect=[0, 0.03, 1, 0.95])
74 | ruta_figura = os.path.join(directorio, "ejercicio_4_resultado.png")
75 | plt.savefig(ruta_figura, dpi=150, bbox_inches='tight')
77 | plt.show()

```

Explicación línea por línea:

- **Línea 48-52:** Muestra en consola las dimensiones de cada componente del proceso.
- **Línea 53-56:** Genera el nombre de archivo de salida (ej. *resultado_ej4_círculo.jpg*) y guarda la imagen con calidad JPG 95%.
- **Línea 58-69:** Asigna en la fila *i* de la matriz de gráficos: Columna 0 (Fondo), Columna 1 (Máscara con mapa gris), Columna 2 (Lena redimensionada) y Columna 3 (Composición final). Oculta ejes con `axis('off')`.
- **Línea 72-77:** Ajusta márgenes evitando superposiciones, guarda la figura completa de 16 paneles a 150 DPI y despliega la ventana interactiva.

3.2 Ejercicio 5 — Separación de Planos R, G, B y Cálculo de Área

Archivo fuente: `ejercicio_5.py` (Código limpio sin comentarios)

Bloque 1: Importaciones y Carga de la Imagen de Neuronas

```
1 | from PIL import Image
2 | import numpy as np
3 | import matplotlib.pyplot as plt
4 | import os
6 | directorio = os.path.dirname(os.path.abspath(__file__))
7 | ruta_imagen = os.path.join(directorio, "fig_05.jpg")
13 | img = Image.open(ruta_imagen).convert("RGB")
```

Explicación línea por línea:

- **Línea 1-4:** Importa Pillow, NumPy (cálculo matricial), Matplotlib y os.
- **Línea 6-7:** Construye la ruta a la imagen de fluorescencia confocal *fig_05.jpg*.
- **Línea 13-17:** Abre la imagen, fuerza formato RGB y reporta en consola resolución y número total de píxeles.

Bloque 2: Separación de Planos Cromáticos R, G, B

```
19 | plano_R, plano_G, plano_B = img.split()
21 | arr_R = np.array(plano_R)
22 | arr_G = np.array(plano_G)
23 | arr_B = np.array(plano_B)
25 | total_pixeles = arr_R.size
```

Explicación línea por línea:

- **Línea 19:** Utiliza el método nativo `img.split()` para descomponer la imagen en tres objetos Image monocromáticos.
- **Línea 21-23:** Convierte cada plano de Pillow en matrices NumPy 2D de enteros no signados de 8 bits (uint8) con valores entre 0 y 255.
- **Línea 25:** Almacena el total de píxeles de la imagen (ancho x alto) mediante el atributo `.size` del array.

Bloque 3: Segmentación por Umbralización y Cálculo de Área

```
27 | umbral = 30
29 | pixeles_R = np.sum(arr_R > umbral)
30 | pixeles_G = np.sum(arr_G > umbral)
31 | pixeles_B = np.sum(arr_B > umbral)
33 | porcentaje_R = (pixeles_R / total_pixeles) * 100
34 | porcentaje_G = (pixeles_G / total_pixeles) * 100
35 | porcentaje_B = (pixeles_B / total_pixeles) * 100
```

Explicación línea por línea:

- **Línea 27:** Establece un umbral de corte de 30 para discriminar la fluorescencia biológica del ruido térmico y fondo oscuro.
- **Línea 29-31:** Crea una máscara booleana por canal (`arr > umbral`) y suma los valores verdaderos (True=1), contando píxeles biológicamente activos.
- **Línea 33-35:** Calcula la proporción porcentual del área cubierta por cada fluoróforo respecto a la superficie completa del sensor.

Bloque 4: Reporte Tabular de Estadísticas de Intensidad

```

37 | print(f"\n{'=' * 55}\n  Análisis de Área por Plano (umbral = {umbral})\n{'=' * 55}")
42 | print(f"  {'Rojo (R)':<12} {pixeles_R:>18,} {porcentaje_R:>11.2f}%")
52 |
for nombre, arr, color in [("Rojo (R)", arr_R, "R"), ("Verde (G)", arr_G, "G"), ("Azul (B)", arr_B, "B")]:
56 |     print(f"      Mínimo:      {arr.min()}")
57 |     print(f"      Máximo:      {arr.max()}")
58 |     print(f"      Media:        {arr.mean():.2f}")
59 |     print(f"      Desv.Std:     {arr.std():.2f}")

```

Explicación línea por línea:

- **Línea 37-47:** Imprime en consola una tabla formateada con alineación de columnas que detalla conteo de píxeles y porcentajes por color.
- **Línea 52-60:** Itera sobre los arrays de cada canal calculando mínimo, máximo, media aritmética (brillo global) y desviación estándar (dispersión de contraste).

Bloque 5: Reconstrucción Tricromática y Visualización Multicanal

```

62 | canal_cero = Image.new("L", img.size, 0)
63 | img_solo_R = Image.merge("RGB", (plano_R, canal_cero, canal_cero))
64 | img_solo_G = Image.merge("RGB", (canal_cero, plano_G, canal_cero))
65 | img_solo_B = Image.merge("RGB", (canal_cero, canal_cero, plano_B))
67 | fig, axes = plt.subplots(2, 4, figsize=(20, 10))
73 | axes[0, 1].imshow(arr_R, cmap='gray')
82 | axes[1, 1].imshow(img_solo_R)
94 | barras = axes[1, 0].bar(canales_nombres, porcentajes, color=['#FF4444', '#44BB44', '#4444FF'])

```

Explicación línea por línea:

- **Línea 62:** Crea una imagen auxiliar 'L' de ceros (negro absoluto) con las mismas dimensiones que la imagen original.
- **Línea 63-65:** Reconstruye imágenes en color puro aislando cada canal mediante `Image.merge('RGB', ...)` y rellenando los otros dos canales con negro.
- **Línea 67-71:** Crea una figura de 2 filas x 4 columnas en Matplotlib.
- **Línea 73-80:** Fila 1: Presenta la imagen original combinada y los tres planos en escala de grises.
- **Línea 82-90:** Fila 2: Presenta las imágenes en color aislado (rojo puro, verde puro, azul puro).
- **Línea 94-110:** Subplot [1, 0]: Genera un gráfico de barras comparativo con el porcentaje de área de cada plano, etiquetando los valores sobre las columnas y guardando la figura.

3.3 Ejercicio 6 — Histograma RGB y Escala de Grises

Archivo fuente: `ejercicio_6.py` (Código limpio sin comentarios)

Bloque 1: Extracción y Segmentación del Histograma de Color

```
1 | from PIL import Image
2 | import numpy as np
3 | import matplotlib.pyplot as plt
4 | import os
13 | img = Image.open(ruta_imagen).convert("RGB")
18 | plano_R, plano_G, plano_B = img.split()
24 | histograma_pil = img.histogram()
26 | hist_R_pil = histograma_pil[0:256]
27 | hist_G_pil = histograma_pil[256:512]
28 | hist_B_pil = histograma_pil[512:768]
```

Explicación línea por línea:

- **Línea 1-4:** Importa librerías requeridas.
- **Línea 13-23:** Carga la imagen de Lena (*fig_00.jpg*), divide sus bandas cromáticas y extrae los arrays NumPy.
- **Línea 24:** Invoca `img.histogram()`: genera una lista contigua de 768 enteros con el conteo de píxeles.
- **Línea 26-28:** Segmenta la lista en tres sublistas de 256 elementos: `[0:256]` para R, `[256:512]` para G y `[512:768]` para B.

Bloque 2: Cálculo de la Moda (Tonalidad Más Frecuente)

```
30 | moda_R = np.argmax(hist_R_pil)
31 | moda_G = np.argmax(hist_G_pil)
32 | moda_B = np.argmax(hist_B_pil)
34 | frecuencia_R = hist_R_pil[moda_R]
35 | frecuencia_G = hist_G_pil[moda_G]
36 | frecuencia_B = hist_B_pil[moda_B]
```

Explicación línea por línea:

- **Línea 30-32:** Aplica `np.argmax()` sobre cada segmento de 256 posiciones: obtiene el índice exacto (0-255) donde la frecuencia es máxima.
- **Línea 34-36:** Accede a dicho índice para obtener el conteo absoluto de píxeles que poseen esa intensidad dominante.

Bloque 3: Conversión a Escala de Grises y Análisis Estadístico

```

52 | img_gris = img.convert("L")
54 | arr_gris = np.array(img_gris)
56 | hist_gris = img_gris.histogram()
58 | moda_gris = np.argmax(hist_gris)
59 | frecuencia_gris = hist_gris[moda_gris]
61 | media_gris = arr_gris.mean()
62 | std_gris = arr_gris.std()
63 | mediana_gris = np.median(arr_gris)

```

Explicación línea por línea:

- **Línea 52:** Convierte la imagen a escala de grises monocromática usando coeficientes de ponderación fisiológica humana ($L = 0.299R + 0.587G + 0.114B$).
- **Línea 54-56:** Convierte a matriz numérica y obtiene su histograma de 256 niveles.
- **Línea 58-59:** Calcula la moda y el pico de frecuencia del canal de luminancia.
- **Línea 61-63:** Calcula métricas estadísticas descriptivas: media (brillo global), desviación estándar (contraste dinámico) y mediana.

Bloque 4: Generación Automática de Conclusiones Analíticas

```

65 | print(f"\n{'=' * 60}\n CONCLUSIONES DEL ANÁLISIS DE HISTOGRAMAS\n{'=' * 60}")
71 | if moda_R > moda_B: print(" -> La imagen tiene predominancia de tonos CÁLIDOS")
76 | brillo_desc = "brillante" if media_gris > 140 else "oscura" if media_gris < 100 else "equilibrada"
81 | contraste_desc = "alto" if std_gris > 60 else "bajo" if std_gris < 35 else "medio/bueno"

```

Explicación línea por línea:

- **Línea 65-70:** Imprime sección de conclusiones calculadas en tiempo de ejecución.
- **Línea 71-75:** Evalúa si la moda roja supera a la azul para clasificar la temperatura de color de la escena (cálida vs fría).
- **Línea 76-80:** Evalúa la media de luminancia para tipificar la exposición fotográfica (sobreexpuesta/subexpuesta/equilibrada).
- **Línea 81-85:** Evalúa la desviación estándar para determinar el aprovechamiento del rango dinámico (contraste alto/medio/bajo).

Bloque 5: Visualización de 9 Paneles y Guardado

```
87 | fig, axes = plt.subplots(3, 3, figsize=(18, 16))
93 | axes[0, 2].barh(["Rojo", "Verde", "Azul", "Gris"], [moda_R, moda_G, moda_B, moda_gris])
102 | axes[1, 0].bar(range(256), hist_R_pil, color='red', alpha=0.7); axes[1, 0].axvline(x=moda_R, color='darkred', linestyle='--')
129 | axes[2, 0].plot(range(256), hist_R_pil, 'r-', label='R'); axes[2, 0].plot(range(256), hist_G_pil, 'g-', label='G'); axes[2, 0].plot(range(256), hist_B_pil, 'b-', label='B')
142 | hist_acumulado = np.cumsum(hist_gris) / total_pixeles
143 | axes[2, 2].plot(range(256), hist_acumulado, 'k-', linewidth=2)
153 | plt.savefig(os.path.join(directorio, "ejercicio_6_resultado.png"), dpi=150, bbox_inches='tight')
```

Explicación línea por línea:

- **Línea 87-90:** Crea una cuadrícula de 3x3 paneles altamente estructurada.
- **Línea 93-100:** Panel [0,2]: Gráfico de barras horizontal comparando las modas de los tres canales y de la escala de grises.
- **Línea 102-126:** Fila 1: Histogramas individuales de R, G y B con barras coloreadas y líneas verticales discontinuas marcando la moda exacta.
- **Línea 129-140:** Panel [2,0]: Histograma RGB superpuesto mostrando las 3 curvas continuas.
- **Línea 142-150:** Panel [2,2]: Curva del histograma acumulado normalizado con `np.cumsum()`, útil para análisis de ecualización.
- **Línea 153-155:** Exporta la figura completa en alta definición y despliega el renderizador.

3.4 Ejercicio 7 — Colorización de Imagen en Escala de Grises

Archivo fuente: `ejercicio_7.py` (Código limpio sin comentarios)

Bloque 1: Configuración y Carga de la Imagen Marina Monocromática

```
1 | from PIL import Image, ImageOps, ImageFilter, ImageEnhance
2 | import numpy as np
3 | import matplotlib.pyplot as plt
4 | import os
6 | directorio = os.path.dirname(os.path.abspath(__file__))
7 | ruta_imagen = os.path.join(directorio, "sea.jpg")
13 | img_gris = Image.open(ruta_imagen).convert("L")
18 | arr_gris = np.array(img_gris, dtype=np.float64)
```

Explicación línea por línea:

- **Línea 1-4:** Importa módulos de Pillow (incluyendo `ImageOps`, `ImageFilter`, `ImageEnhance`), NumPy y Matplotlib.
- **Línea 6-7:** Localiza la imagen marina en escala de grises *sea.jpg*.
- **Línea 13-16:** Carga y asegura conversión a modo 'L' de un solo canal.
- **Línea 18:** Convierte los píxeles a array NumPy en coma flotante de 64 bits (`np.float64`) para cálculos de precisión continua sin desbordamiento.

Bloque 2: Método 1: Mapa de Colores Matemático Personalizado (Océano)

```
22 | arr_normalizado = arr_gris / 255.0
24 | canal_R = np.clip(arr_normalizado * 0.3 + np.power(arr_normalizado, 3) * 0.7, 0, 1)
26 | canal_G = np.clip(arr_normalizado * 0.5 + np.power(arr_normalizado, 2) * 0.5, 0, 1)
28 | canal_B = np.clip(0.15 + arr_normalizado * 0.85, 0, 1)
30 | arr_color_oceano = np.stack([
31 |     (canal_R * 255).astype(np.uint8),
32 |     (canal_G * 255).astype(np.uint8),
33 |     (canal_B * 255).astype(np.uint8)
34 | ], axis=-1)
36 | img_oceano = Image.fromarray(arr_color_oceano, mode="RGB")
```

Explicación línea por línea:

- **Línea 22:** Normaliza las intensidades al rango unitario $[0.0, 1.0]$.
- **Línea 24:** Canal Rojo: Curva fuertemente comprimida mediante potencia cúbica (conserva el rojo solo en la espuma más brillante).
- **Línea 26:** Canal Verde: Curva cuadrática intermedia para tonalidades turquesa y esmeralda.
- **Línea 28:** Canal Azul: Elevación de base (bias = 0.15) asegurando presencia de azul profundo incluso en las sombras más oscuras.
- **Línea 30-34:** Multiplica por 255, castea a `uint8` y apila los tres canales a lo largo del último eje (eje z de color).
- **Línea 36:** Convierte el tensor NumPy resultante de vuelta a objeto Pillow RGB.

Bloque 3: Método 2: Tabla de Búsqueda LUT (Efecto Atardecer Cálido)

```
39 | lut_R = [min(255, int(i * 1.3 + 20)) for i in range(256)]
40 | lut_G = [min(255, int(i * 0.85)) for i in range(256)]
41 | lut_B = [min(255, int(i * 0.55)) for i in range(256)]
43 | lut_total = lut_R + lut_G + lut_B
45 | img_gris_rgb = img_gris.convert("RGB")
46 | img_atardecer = img_gris_rgb.point(lut_total)
```

Explicación línea por línea:

- **Línea 39-41:** Genera tres tablas de búsqueda (LUT) de 256 elementos: Rojo potenciado (+30% más base de 20), Verde atenuado (-15%) y Azul fuertemente filtrado (-45%).
- **Línea 43:** Concatena las 3 listas en una única LUT de 768 enteros.
- **Línea 45:** Replica el canal gris en tres canales idénticos RGB.
- **Línea 46:** Aplica `img.point(lut_total)`: transforma cada píxel instantáneamente mediante mapeo directo por tabla en C, logrando un atardecer dorado cálido.

Bloque 4: Método 3: Paleta Científica de Matplotlib ('ocean')

```
48 | cmap_ocean = plt.colormaps.get_cmap('ocean')
50 | arr_normalizado_2 = np.array(img_gris, dtype=np.float64) / 255.0
52 | arr_rgba = cmap_ocean(arr_normalizado_2)
54 | arr_rgb = (arr_rgba[:, :, :3] * 255).astype(np.uint8)
56 | img_tropical = Image.fromarray(arr_rgb, mode="RGB")
```

Explicación línea por línea:

- **Línea 48:** Obtiene el colormap científico predefinido 'ocean' de Matplotlib.
- **Línea 50:** Normaliza los valores de la imagen en escala de grises a float entre 0.0 y 1.0.
- **Línea 52:** Aplica el colormap sobre la matriz: devuelve un array RGBA de 4 canales flotantes.
- **Línea 54-56:** Descarta el canal alfa (`[:, :, :3]`), escala a 255, convierte a uint8 y crea la imagen RGB final.

Bloque 5: Método 4: Mapeo Tricromático con Postprocesamiento (Realista)

```
58 | color_sombras = (5, 25, 60)
59 | color_medios = (20, 120, 160)
60 | color_luces = (220, 240, 255)
62 | img_realista = ImageOps.colorize(img_gris, black=color_sombras, white=color_luces, mid=color_medios)
64 | img_realista = img_realista.filter(ImageFilter.SMOOTH)
66 | realzador = ImageEnhance.Color(img_realista)
67 | img_realista = realzador.enhance(1.3)
```

Explicación línea por línea:

- **Línea 58-60:** Define la triada de color cromática: azul marino profundo en sombras, verde azulado marino en medios tonos y blanco espumoso con tinte celeste en luces.
- **Línea 62:** Ejecuta `ImageOps.colorize` para interpolar linealmente los niveles de gris entre las tres tonalidades objetivo.
- **Línea 64:** Aplica `ImageFilter.SMOOTH` para homogenizar transiciones tonales en el oleaje.
- **Línea 66-67:** Aumenta la viveza y saturación cromática en un 30% mediante `ImageEnhance.Color.enhance(1.3)`.

Bloque 6: Comparación Multipanel y Exportación Final

```

70 | fig, axes = plt.subplots(2, 3, figsize=(18, 12))
76 | axes[0, 0].imshow(img_gris, cmap='gray'); axes[0, 0].set_title("Original: sea.jpg (Grises)")
80 | axes[0, 1].imshow(img_oceano); axes[0, 1].set_title("Método 1: Océano (NumPy)")
84 | axes[0, 2].imshow(img_atardecer); axes[0, 2].set_title("Método 2: Atardecer (LUT)")
88 | axes[1, 0].imshow(img_tropical); axes[1, 0].set_title("Método 3: Tropical (Colormap)")
92 | axes[1, 1].imshow(img_realista); axes[1, 1].set_title("Método 4: Realista (ImageOps)")
96 | im_h = img_gris.size[1]; ancho_seccion = img_gris.size[0] // 3
97 | arr_comp = np.zeros((im_h, ancho_seccion * 3, 3), dtype=np.uint8)
98 | arr_comp[:, :ancho_seccion] = np.array(img_gris.convert('RGB'))[:, :ancho_seccion]
99 | arr_comp[:, ancho_seccion:2*ancho_seccion] = np.array(img_oceano[:, ancho_seccion:2*ancho_seccion])
100 | arr_comp[:, 2*ancho_seccion:] = np.array(img_realista[:, 2*ancho_seccion:])
101 | axes[1, 2].imshow(arr_comp); axes[1, 2].set_title("Comparación: Grises | Océano | Realista")
111 | plt.savefig(os.path.join(directorio, "ejercicio_7_resultado.png"), dpi=150, bbox_inches='tight')

```

Explicación línea por línea:

- **Línea 70-75:** Configura una cuadrícula de 2 filas x 3 columnas con títulos estilizados.
- **Línea 76-95:** Representa la imagen original en escala de grises y cada una de las cuatro interpretaciones cromáticas.
- **Línea 96-105:** Construye una composición trifásica lado a lado (Original en grises, Océano matemático y Realista) en un solo panel para comparación directa de fidelidad.
- **Línea 111-115:** Exporta la figura final a resolución completa y despliega el visualizador.